

Volume 40

Sentinel Web Security Engine (SWSE)

Status: Core Security Domain

Priority: Critical

Vision

The Web Security Engine is responsible for continuously assessing the security posture of authorized web applications, APIs, internet-facing services, and supporting infrastructure.

Unlike traditional scanners, the engine does not simply generate vulnerability lists.

Its purpose is to build evidence that contributes to the Digital Twin.

Mission

The mission is not:

Scan website

The mission is:

Understand the complete security posture of this web service.

Supported Assets

The engine manages objects such as:

- Websites
- Web Applications
- REST APIs
- GraphQL APIs
- Reverse Proxies
- Load Balancers
- DNS Records
- Certificates
- CDN Endpoints
- Authentication Services
- Identity Providers
- Web Servers

Every asset becomes a Sentinel Object.

Web Object Model

Organization

Everything is connected.

Web Mission Types

The engine supports reusable mission templates.

Discovery Mission

Purpose:

Discover:

- Domains
 - Subdomains
 - Services
 - Certificates
 - APIs
-

Configuration Mission

Collect evidence about:

- HTTP configuration
 - HTTPS
 - TLS
 - DNS
 - Cookies
 - Headers
-

Security Header Mission

Verify:

- CSP
- HSTS
- X-Frame-Options
- Referrer Policy
- Permissions Policy
- X-Content-Type-Options

Evidence only.

Certificate Mission

Collect:

- Expiration
 - Trust chain
 - Issuer
 - Key size
 - Signature algorithm
 - SAN entries
-

Authentication Mission

Verify:

- MFA support (where applicable)
 - Session handling
 - Cookie configuration
 - Secure attributes
 - Logout behavior
 - Token configuration
-

API Mission

Evaluate authorized APIs for:

- Documentation exposure
 - Authentication configuration
 - Versioning
 - Transport security
 - Error handling
 - Rate limiting configuration (where observable)
-

Continuous Monitoring Mission

Observe changes such as:

- New certificates
 - DNS updates
 - New services
 - Header changes
 - Configuration drift
-

Evidence Types

The engine produces structured evidence, including:

- TLS configuration
- Certificate metadata
- HTTP response headers
- Redirect behavior
- DNS records
- Cookie attributes
- Response metadata
- Observed authentication mechanisms
- API endpoint metadata (within authorized scope)

The Rule Engine interprets this evidence.

Website Health Model

Each website has measurable dimensions.

Examples:

Transport Security

Authentication

Certificate Health

Configuration

Availability

Verification

Trust

Evidence Freshness

No single metric tells the whole story.

Website Timeline

Every change is preserved.

Website Created

Digital Twin Integration

Every website contributes to the Digital Twin.

Relationships include:

Website

↓

API

↓

Identity Provider

↓

Certificate

↓

DNS

↓

Cloud

↓

Owner

↓

Business Service

This helps assess operational impact.

Web Trust Engine

The Trust Engine consumes verified evidence.

Examples of positive indicators:

- Modern TLS configuration.
- Current certificates.
- Strong security headers.
- Verified ownership.
- Regular assessments.

Trust changes only when supported by evidence.

Compliance Mapping

Evidence can be mapped to organizational controls and frameworks such as:

- ISO 27001
- CIS Controls
- OWASP ASVS (where applicable)
- Internal security baselines

Mappings remain transparent and versioned.

Web Dashboard

The dashboard answers:

- Which sites have changed?
- Which certificates expire soon?
- Which services have not been verified recently?
- Which business-critical applications require attention?
- Which missions are active?

No alert overload.

Home User Mode

A personal website owner should see:

- Overall health.
- HTTPS status.
- Certificate validity.
- Domain configuration summary.
- Recommended improvements.
- Verification history.

Plain language.

Enterprise Mode

Large organizations can organize websites by:

Business Unit

↓

Application

↓

Environment

↓

Owner

↓

Criticality

↓

Compliance

This supports governance at scale.

NEW CORE ENGINE

Configuration Drift Engine

One of the most valuable enterprise capabilities.

Continuously detects changes such as:

- TLS configuration.
- HTTP headers.
- Certificates.
- DNS.
- Reverse proxy configuration.
- Security policies.

Unexpected changes generate review tasks rather than assumptions.

NEW CORE ENGINE

Internet Exposure Engine

Maintains an inventory of authorized internet-facing assets.

Examples:

- Domains.
- Subdomains.
- Public APIs.
- Public services.
- Certificates.

This provides continuous visibility into the organization's approved external footprint.

NEW CORE ENGINE

Web Relationship Engine

Maintains relationships between:

- Domains.
- APIs.
- Certificates.
- Cloud resources.
- Owners.
- Business processes.

Everything becomes part of the Knowledge Graph.

NEW IDEA

Website Security Passport

Every website receives a continuously updated passport.

Example:

Website

A concise operational summary backed by detailed evidence.

NEW IDEA

Change Journal

Instead of static reports.

Every website has a living journal.

Example:

Monday

Certificate renewed.

Tuesday

New API deployed.

Wednesday

Header configuration updated.

Thursday

Verification completed.

Friday

Trust increased.

This provides context rather than isolated findings.

NEW IDEA

Web Mission Replay

Every authorized assessment can be replayed.

Users can inspect:

- Evidence collection sequence.
- Configuration changes.
- Rule evaluations.
- Trust updates.
- Recommendations.

This supports learning, auditing, and troubleshooting.

PS-ADR-0035

Evidence-Driven Web Security

Status: Accepted

Decision

The Web Security Engine shall collect structured evidence about authorized web assets and delegate interpretation to the Rule Engine, Trust Engine, and Decision Intelligence Engine.

Rationale

Separating observation from evaluation improves explainability, reuse, testing, and long-term maintainability.

NEW LONG-TERM VISION

The Living Web

I want to introduce a concept that reflects our overall philosophy:

A website is **not** a static endpoint.

It is a living object that evolves through deployments, certificate renewals, infrastructure changes, policy updates, and business decisions.

Project Sentinel should therefore treat every website as a continuously evolving component of the organization's Digital Twin rather than as a target for occasional scans.

A New Engineering Principle

I'd like to formalize another rule before we begin implementing the remaining security domains:

Every security engine in Project Sentinel shall follow the same architecture:

- Mission-first.
- Object-centric.
- Evidence-driven.
- Event-based.
- Trust-aware.
- Workflow-enabled.
- Explainable.
- Accessible (WCAG).
- Localizable (Norwegian, English, and additional languages).
- Extensible through documented interfaces.

This consistency will make the platform easier to learn, easier to maintain, and easier to extend over the coming decades.

Looking Ahead

The next engineering volume will be **Volume 41 - Sentinel Endpoint Security Engine**, where we define the complete architecture for protecting Windows, Linux, and macOS systems—from home computers to enterprise fleets. This is also where we will specify how Project Sentinel detects configuration drift, evaluates endpoint health, identifies evidence of compromise, and guides users through remediation using the same evidence-first, explainable model we've established throughout the platform.

====[USER]=====

Good and continue

====[ASSISTANT]=====

Excellent. Now we are entering one of the largest engineering volumes of the entire project.

This module will probably become one of the biggest competitive advantages of Project Sentinel because **every organization—from private users to governments—depends on endpoint security.**

Our goal is **not** to build another antivirus.

Our goal is to build the **most understandable Endpoint Security Engine ever designed.**

PROJECT SENTINEL BLUEPRINT

Volume 41